CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

DANILO IGLESIA DA SILVA

MÁRIO HENRIQUE AKIHIKO DA COSTA ADANIYA

SISTEMA DE ROTAS INTELIGENTES

Londrina

2026

SUMÁRIO

1 INTRODUÇÃO 2
1.1 Proposta e Objetivos 2
1.1.1 Objetivos específicos 3
1.2 Justificativa 3

2 DIAGRAMAS 4
2.1 Diagrama de Caso de Uso 4
2.2 Diagrama de Classes 5
2.3 Diagrama Entidade Relacionamento 6
2.4 Diagrama de Implantação 7
2.5 Diagrama de Sequência 8

3 WORKFLOW (AS IS) NA NOTAÇÃO BPMN 9

4 RECURSOS E AMBIENTE DE DESENOVOLVIMENTO 10
4.1 Linguagem de Programação e Framework 10
4.2 Banco de dados 10
4.3 FERRAMENTAS DE MAPEAMENTO E GEOLOCALIZAÇÃO 10
4.4 Ambiente de Desenvolvimento Integrado (IDE) 11
4.5 Controle de Versão 11

REFERÊNCIAS 12

1 INTRODUÇÃO

O presente plano de estágio tem como objetivo documentar o desenvolvi mento do S.R.I.- Sistema de Rotas Inteligentes, um aplicativo mobile projetado para otimizar a criação de rotas ponto a ponto e circuitos, com foco em motoristas autôno mos, entregadores e pequenas empresas de logística. No contexto atual da mobilidade urbana existem diversos desafios, como au mento do trânsito nas grandes cidades, o consumo elevado de combustível e a ne cessidade crescente de entregas rápidas. Atualmente, os motoristas dependem de soluções generalistas como Google Maps ou Waze, que, apesar de úteis, não ofere cemrecursos especializados para o planejamento de rotas complexas, como rotas em circuito que retornem ao ponto incial ou que possam ser salvas para uso recorrente. Essa falta de soluções voltadas especificamente para entregadores e peque nas de empresas de logística gera ineficiência operacional, elevando custos e redu zindo a produtividade. Conforme identificado no levantamento realizados através de pesquisas, as principais incluem: • Dificuldade na criação de rotas de personalizadas que atendam às necessidades específicas de entregadores. Necessidade de planejar trajetos manualmente, ponto a ponto, em cada entrega. • Falta de integração entre planejamento das rotas e a execução prática durante o deslocamento.

O S.R.I. busca resolver estes problemas ao fornecer uma plataforma mobile intuitiva, que permite criar rotas personalizadas, salvar trajetos frequentes e otimizar o tempo de deslocamento, promovendo maior eficiência e economia. O sistema utilizará APIs de geolocalização(Google Maps Platform ou TomTom). recursos de GPS em tempo real, este projeto será desenvolvido em Flutter, com banco de dados lcoal em SQLite, oferecendo compatilidade com dispositivos Andriod. Este plano de estágio segue a metodologio baseada no RUP (Rational Unified Process), contemplando todas as etapas do projeto: Levantamento de requisitos, planajamento, desenvolvimento e implantação. O documento também visa descrever o ambiente de desenvolvimento, cronograma, recursos utiliados e justificativas para a execução deste projeto.

1.1 PROPOSTA E OBJETIVOS

A proposta do S.R.I. é desenvolver um aplicativo mobile que possibilite ao usuário criar rotas inteligentes personalizadas, incluido rotas em circuito, além de salvar trajetos frequentes para reutilização. O sistema será simples, acessível e focado na realidade de entregadores e motoristas que necessitam otimizar seu tempo e reduzir custos operacionais.

Com isso, o aplicativo irá suprir uma lacuna de mercado, oferecendo funcio-

nalidades que não estão disponíveis em ferramentas tradicionais.

Desenvolver um aplicativo mobile para criação e gerenciamento de rotas inteligentes proporcionando maior eficiência no deslocamento de entregadores e motoristas autônomos.

1.1.1 Objetivos específicos

- Implementar um módulo de CRUD completo para pontos de interesse, utilizando a biblioteca sqflite para persistência local e garantindo a visualização imediata dos marcadores no mapa após o cadastro.
- Implementar a funcionalidade de rotas em circuito
- Possibilitar o armazenamento de rotas salvas para reutilização.
- Garantir um tempo de respostas inferior a 30 segundos no cálculo das rotas.
- Proporcionar uma interface amigável, adaptada para uso em movimento.
- Integrar API's de geolocalização para cálculo otimizado de trajetos.

1.2 JUSTIFICATIVA

Como crescimento do mercado de entregas e logística, há uma demanda por ferramentas que auxiliem no planejamento de rotas. Atualmente, motoristas e entrega dores utilizam soluções generalistas que não atendem às necessidades específicas do setor, o que resulta em tempo perdido, gasto com combustível e baixa produtividade. O S.R.I. se justifica como uma solução inovadora por:

- Foco no público-alvo: voltado exclusivamento para entregadores e pequenas empresas de logística.
- Funcionalidade personalizadas: Criação de rotas em circuito e salvamento de trajetos.
- Eficiência operacional: Redução do tempo de planejamento e execução de rotas.
- Acessibilidade: Aplicativo gratuito, desenvolvido com ferramentas oper source, visando reduzir custos de implantação. (REDHAT, 2024)

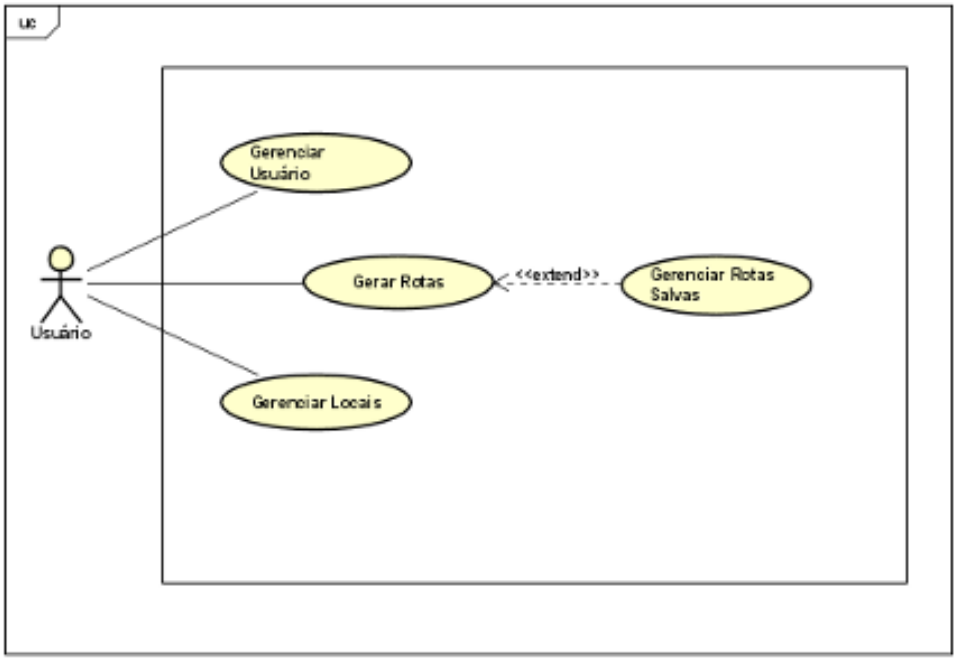
O projeto será desenvolvido como uma etapa prática do estágio obirgatório, permitindo a aplicação dos conhecimentos adquiridos ao longo do curso e buscando gerar impacto na rotina dos usuários.

2 DIAGRAMAS

2.1 DIAGRAMA DE CASO DE USO

Um diagrama de caso de uso é uma representação gráfica das interações entre atores externos e um sistema de software. Ele descreve os diferentes casos de uso e as relações entre esses casos e os atores. Os elementos principais incluem atores, casos de uso e relacionamentos entre eles. É usado para capturar e comunicar os requisitos funcionais de um sistema de forma clara e compreensível como pode ser verificado na Figura 1.

Figura 1 – Caso de Uso



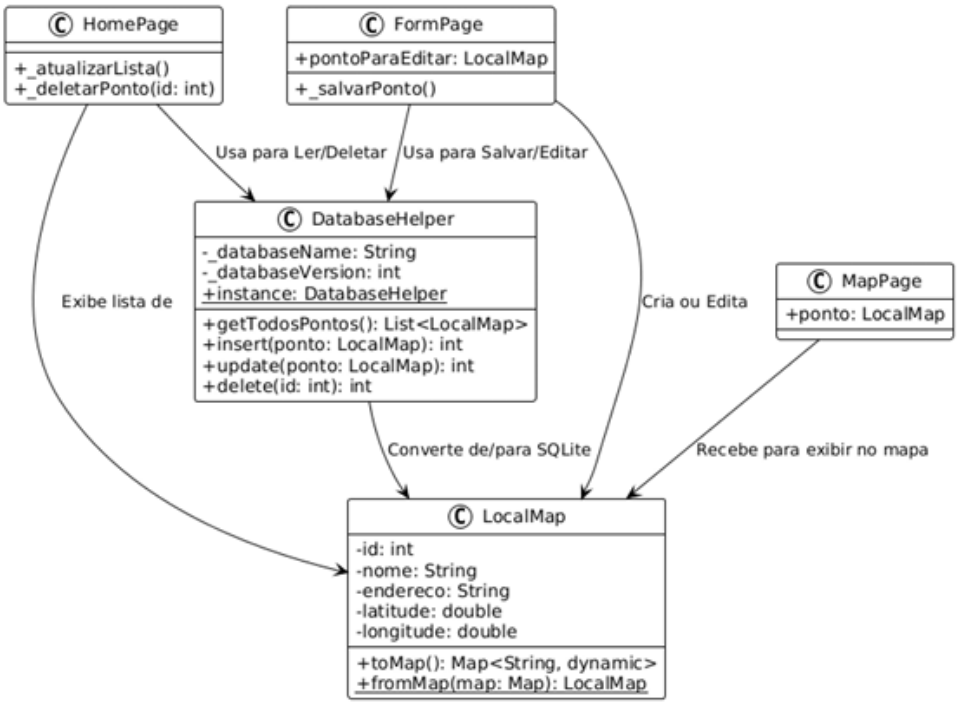
Conforme detalhado na Figura 1, o ator Usuário será o único a utilizar o sistema.

1. **Gerenciar Usuário**
    - Faz todo o CRUD da conta do Usuário
2. **Gerar rotas**
    - Faz todo o CRUD dos locais, este será o caso utilizado na pré-banca.
3. **Gerenciar rotas salva**
    - Registra as rotas feitas, realiza o crud das mesmas.

2.2 DIAGRAMA DE CLASSES

Um diagrama de classe representa visualmente a descrição das classes de um sistema, incluindo seus atributos, métodos e relacionamentos. Utilizado na engenharia de software, ele facilita a compreensão e comunicação da estrutura e comportamento do sistema, como visto na Figura 2.

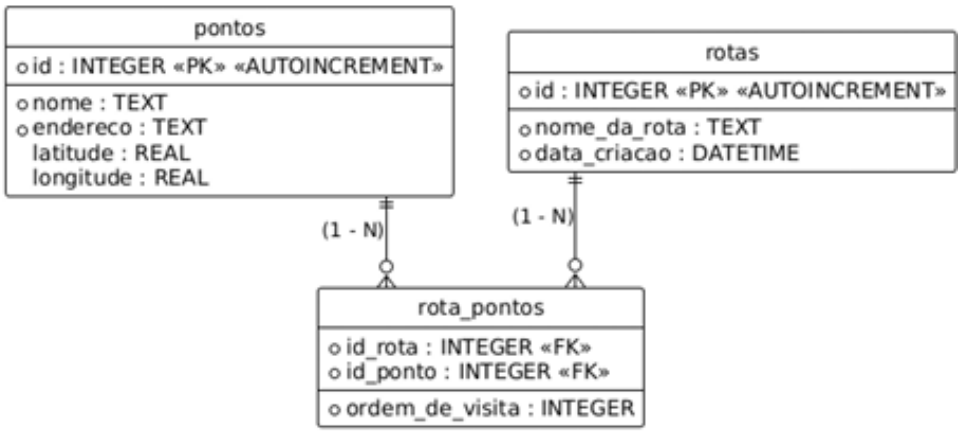
Figura 2 – Diagrama de Classe



2.3 DIAGRAMA ENTIDADE RELACIONAMENTO

Um diagrama de entidade e relacionamento (DER) é essencial para o design de bancos de dados, mostrando as entidades de um sistema e os relacionamentos entre elas. Ele mapeia a estrutura e as interações dos dados dentro do banco. Os componentes principais de um DER incluem entidades, atributos e relacionamentos, clarificando como os dados se conectam e se organizam no banco de dados, representado abaixo na Figura 3.

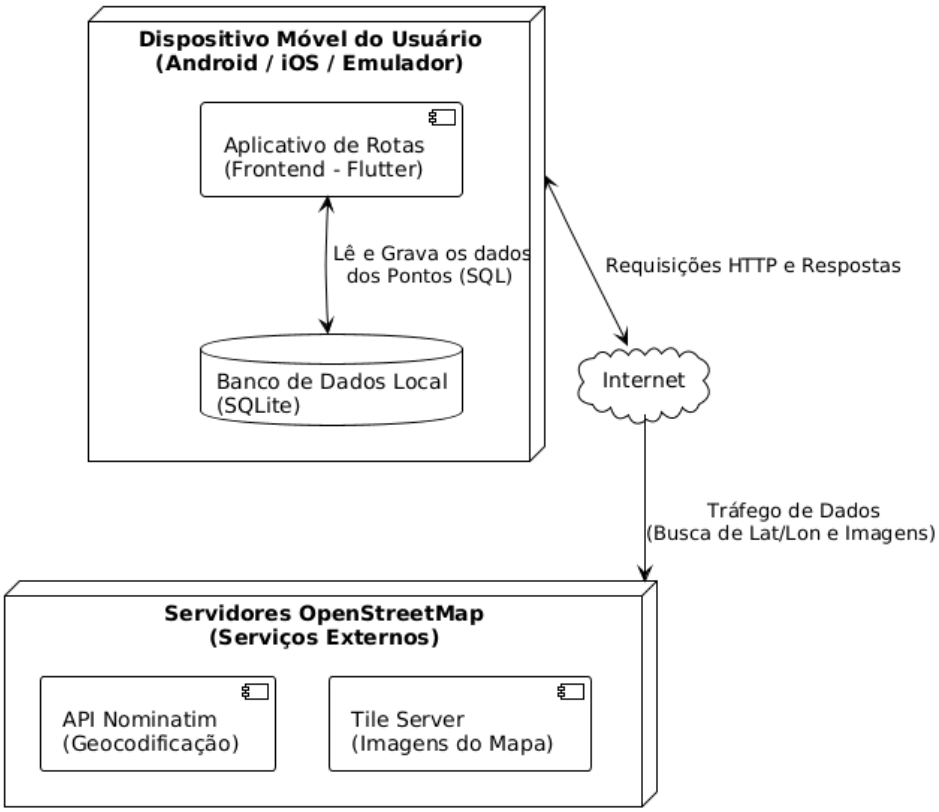
Figura 3 – Diagrama Entidade Relacionamento



2.4 DIAGRAMA DE IMPLANTAÇÃO

O diagrama de implantação, mostra a configuração física dos elementos de hardware e software em um sistema. Ele detalha como os componentes de software são distribuídos nos nós de hardware e suas interações. Esse diagrama ajuda a visu alizar a arquitetura do sistema e a entender a relação entre os componentes físicos e lógicos, como se pode observar na Figura 4.

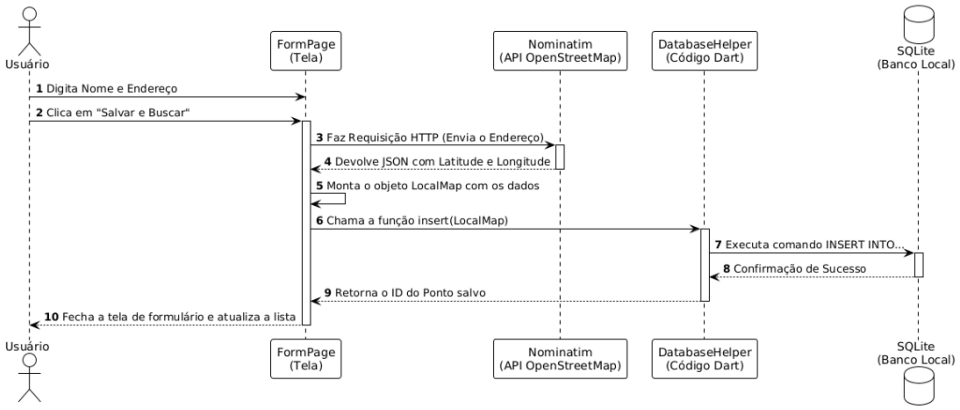
Figura 4 – Diagrama de Implantação



2.5 DIAGRAMA DE SEQUÊNCIA

Diagrama de sequência é uma representação gráfica, utilizada para ilustrar a interação entre objetos em um sistema ao longo do tempo. Ele mostra a ordem das mensagens trocadas entre os objetos para realizar uma determinada função ou processo. Mostrado na Figura 5, esse diagrama é útil para entender e documentar a dinâmica do sistema.

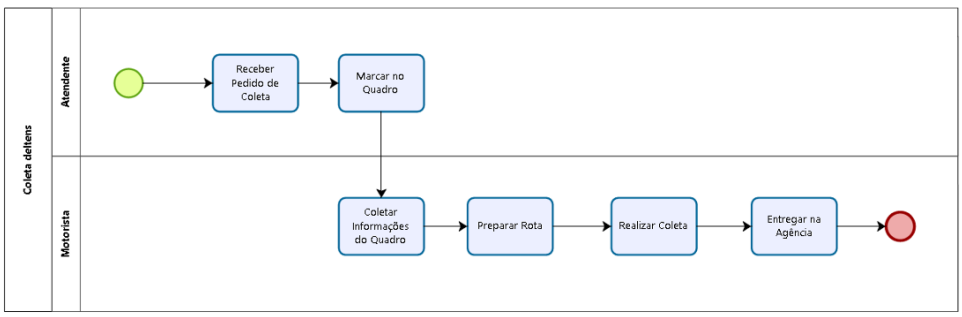
Figura 5 – Diagrama de Sequência



3 WORKFLOW (AS IS) NA NOTAÇÃO BPMN

Abaixo na Figura 6, mostra um fluxograma, de todo o processo de um estabelecimento, onde um processo de rota é feito de forma manual, que no caso seria como é realizado agora.

Figura 6 – Workflow AS IS



4 RECURSOS E AMBIENTE DE DESENOVOLVIMENTO

"Recursos e ambiente de desenvolvimento"refere-se ao conjunto de ferramentas, tecnologias e recursos disponíveis para os desenvolvedores de software durante o processo de criação de um aplicativo, sistema ou projeto de software.

(MILETTO; BERTAGNOLLI, 2014)

4.1 LINGUAGEM DE PROGRAMAÇÃO E FRAMEWORK

A linguagem usada no desenvolvimento do aplicativo mobile será Dart (DART, 2024), operando em conjunto com o framework Flutter (FLUTTER, 2024). Essa escolha foi feita devido à capacidade do Flutter de compilar o código nativamente para múltiplas plataformas (Android e iOS) a partir de um único código-base, garantindo alta performance e uma interface de usuário fluida e responsiva para a criação e visualização dos pontos no mapa.

- **Dart**: versão 3.41.5
- **Flutter**: versão 85.3.1

4.2 BANCO DE DADOS

Para o armazenamento local, será utilizado o banco de dados SQLite (SQLITE, 2024), integrado ao Flutter através do pacote sqflite. A escolha pelo SQLite justifica-se por ser um banco de dados relacional leve, embutido diretamente no dispositivo móvel, que permite a operação offline do aplicativo. Sua estrutura relacional será essencial para os passos futuros do projeto, garantindo estabilidade e escalabilidade ao relacionar os pontos isolados para a criação de circuitos e rotas complexas.

- **sqflite**: versão 2.4.2+1

4.3 FERRAMENTAS DE MAPEAMENTO E GEOLOCALIZAÇÃO

A exibição visual dos mapas e a conversão de endereços em coordenadas geográficas (geocodificação) serão realizadas utilizando os serviços do OpenStreetMap (OPENSTREETMAP, 2024). Através da API gratuita Nominatim e do pacote "flutter.map", o sistema garante a busca de locais via texto e a renderização de mapas em

blocos (tiles) sem a necessidade de chaves de API pagas ou dependência de serviços fechados, promovendo o uso de tecnologias de código aberto.

- **OPENSTREETMAP**: versão 5.3.2 (OPENSTREETMAP, 2024)

4.4 AMBIENTE DE DESENVOLVIMENTO INTEGRADO (IDE)

O ambiente principal usado para escrever e estruturar o código do sistema será o Visual Studio Code (MICROSOFT, 2024b). Ele atua como uma plataforma leve, versátil e altamente customizável através de extensões específicas para Flutter e Dart. Em conjunto, utilizaremos as ferramentas do Android Studio (MICROSOFT, 2024a) exclusivamente para o fornecimento do emulador de dispositivo virtual (Android Virtual Device), permitindo testes fidedignos e em tempo real do aplicativo durante o desenvolvimento.

- **Visual Studio Code**: versão 1.87.2
- **Android Studio**: versão 2025.3.4

4.5 CONTROLE DE VERSÃO

Git para gerenciar o código fonte em conjunto com a plataforma GitHub.

REFERÊNCIAS

DART. *Dart programming language*. [S.l.], 2024. 10

FLUTTER. *Flutter: Build apps for any screen.* [S.l.], 2024. 10

MICROSOFT. *Android Studio: The official IDE for Android*. [S.l.], 2024. 11

MICROSOFT. *Visual Studio Code: Code Editing. Redefined.* [S.l.], 2024. 11

MILETTO, E. M.; BERTAGNOLLI, S. de C. *Desenvolvimento de Software II: Introdução ao Desenvolvimento Web com HTML, CSS, JavaScript e PHP-Eixo: Informação e Comunicação-Série Tekne*. [S.l.]: Bookman Editora, 2014. 10

OPENSTREETMAP. *OpenStreetMap: The Free Wiki World Map.* [S.l.], 2024. 10, 11

SQLITE. *SQLite Home Page.* [S.l.], 2024. 10